Collaborative Document Editor with AI Writing Assistant

Requirements Engineering, Architecture, Project Management, and Proof of Concept

Project Codename:	CollabWrite
Assignment:	Midterm Project
Prepared for:	February 2026 brief
Prepared by:	Ahmed Aymna
	Mohamed Ibrahim
	Aya
	Praveen Anilkumar Shukla
Prepared on:	April 3, 2026

This report documents the requirements, architecture, repository organization, management approach, and runnable proof of concept for a real-time collaborative document editor
with an integrated AI writing assistant.

Contents

1 Requirements Engineering **3**
1.1 Stakeholder Analysis . . . 3
1.2 Functional Requirements . . . 4
1.3 Non-Functional Requirements . . . 8
1.4 User Stories and Scenarios . . . 9
1.5 Requirements Traceability Matrix . . . 11

2 System Architecture **12**
2.1 Architectural Drivers . . . 12
2.2 System Design Using the C4 Model . . . 12
2.2.1 Level 1: System Context Diagram . . . 13
2.2.2 Level 2: Container Diagram . . . 14
2.2.3 Level 3: Component Diagram for the Application API . . . 16
2.2.4 Feature Decomposition . . . 17
2.3 AI Integration Design . . . 18
2.3.1 Context and Scope . . . 18
2.3.2 Trustworthiness of Cloud-Based Completion . . . 18
2.3.3 Suggestion UX . . . 18
2.3.4 AI During Collaboration . . . 21
2.3.5 Prompt Design . . . 21
2.3.6 Model and Cost Strategy . . . 21
2.4 API Design . . . 21
2.4.1 API Style . . . 21
2.4.2 REST API Contract . . . 21
2.4.3 Realtime Session Contract . . . 24
2.4.4 Long-Running AI Operations . . . 24
2.4.5 Error Handling Strategy . . . 24
2.5 Authentication and Authorization . . . 24
2.6 Communication Model . . . 25
2.7 Code Structure and Repository Organization . . . 25
2.7.1 Monorepo vs. Multi-Repo . . . 25
2.7.2 Directory Layout . . . 25
2.7.3 Shared Code . . . 26

2.7.4 Configuration Management . . . 26
2.7.5 Testing Structure . . . 26
2.8 Data Model . . . 27
2.9 Architecture Decision Records . . . 28
2.9.1 ADR-01: Monorepo with Shared Contract Package . . . 28
2.9.2 ADR-02: REST for CRUD plus WebSocket for Collaboration . . . 28
2.9.3 ADR-03: AI Suggestions Are Proposals, Not Immediate Mutations . . . 28
2.9.4 ADR-04: Repository Abstraction with In-Memory PoC Adapter . . . 29

3 Project Management and Team Collaboration 30
3.1 Team Structure and Ownership . . . 30
3.2 Development Workflow . . . 30
3.3 Development Methodology . . . 31
3.4 Risk Assessment . . . 31
3.5 Timeline and Milestones . . . 31

4 Proof of Concept 33
4.1 What the PoC Demonstrates . . . 33
4.2 What the PoC Intentionally Does Not Implement Yet . . . 33
4.3 Architecture-to-Code Alignment . . . 33
4.4 How to Run the PoC . . . 34
4.5 Verification Evidence . . . 34

A Mermaid Source Listings 35
A.1 System Context Diagram . . . 35
A.2 Container Diagram . . . 35
A.3 Backend Component Diagram . . . 36
A.4 Data Model Diagram . . . 37
A.5 AI Lifecycle Diagram . . . 39

1 Requirements Engineering

1.1 Stakeholder Analysis

Table 1: Stakeholder analysis

Stakeholder	Goals	Concerns	Influence on Requirements
Startup founders / product owner	Ship a credible, easy-to-use MVP quickly, differentiate through AI, and keep operating cost under control	Weak UX, slow delivery, privacy incidents, and runaway AI spend can damage adoption and margin	Drives MVP-first scope, meaningful AI value, graceful degradation when AI is unavailable, and cost-aware model strategy
Team leads / document owners	Coordinate contributors, preserve quality, review changes, and recover from mistakes	Unclear ownership, accidental overwrites, weak version history, and no accountability for AI edits	Drives versioning, revert workflows, tracked AI interactions, and owner-managed sharing controls
Data security and privacy team	Protect sensitive content during editing, storage, sharing, and AI processing	Weak permission checks, insecure sessions, excessive AI context, unsafe retention, and policy-violating prompts	Drives backend-enforced authorization, scoped AI context, encryption, retention limits, request validation, and timeout behavior
Organization IT administrators	Manage users, onboarding and offboarding, and workspace-wide sharing plus AI policy at scale	Over-permissive sharing, stale sessions after removal, incorrect role assignment, and weak auditability	Drives RBAC, centralized permission management, session revocation expectations, link-sharing policy, and audit-trail requirements
DevOps / platform operations	Keep the platform stable, observable, and scalable under bursts of realtime collaboration and AI traffic	WebSocket fan-out saturation, edit storms, AI backlog spillover, and poor incident visibility	Drives latency and availability targets, rate limiting, structured logs, metrics, tracing, and isolation of AI or export workers from core editing
Developers / maintainers	Build features in parallel, keep contracts stable, and test safely	Tight coupling between web and API, schema drift, and merge conflicts across features	Drives the monorepo choice, shared contract package, module decomposition, ADRs, and automated testing strategy

1.2 Functional Requirements

The functional specification is organized around the four capability areas from the brief: real-time collaboration, AI writing assistance, document management, and user management. Each requirement below includes a unique identifier, the triggering condition, expected system behavior, and acceptance criteria.

Table 2: Functional requirements specification

Area	ID	Level	Description	Trigger	Expected system behavior	Acceptance criteria
Real-time collaboration	COLL-1	Capability	The system shall support synchronous multi-user editing of the same document.	Two or more authorized collaborators open the same document.	The system establishes a shared editing session and keeps participants on the latest committed content version.	Two browsers connected to the same document see the same latest content after each committed update.
Real-time collaboration	COLL-1.1	Sub-requirement	The system shall propagate collaborator edits to active participants through push-based communication.	An editor submits a live document change.	The server validates access, stores the new content, increments the version, and broadcasts the update to the session.	An edit made by User A appears for User B without page refresh and includes an updated version number.
Real-time collaboration	COLL-1.2	Sub-requirement	The system shall expose presence awareness showing who is currently in the document and each user's latest cursor index.	A participant joins, leaves, or moves their cursor.	The system updates session presence and broadcasts the current participant list.	Active participants can identify who is online and see cursor positions update in the shared session.
Real-time collaboration	COLL-1.3	Sub-requirement	The system shall detect stale edits and prevent silent overwrite when a change is based on an out-of-date version.	A client submits an edit with a stale base version.	The server rejects the mutation, returns a conflict error, and the client refreshes to the latest snapshot.	A stale update returns `VERSION_CONFLICT` and the client can recover by reloading the latest state.
AI writing assistant	AI-1	Capability	The system shall provide role-aware AI assistance for rewriting, summarizing, translating, and restructuring selected text.	An authorized user invokes an AI feature.	The system validates AI permissions, captures scoped context, records the request, and returns a suggestion asynchronously.	An authorized user can request any supported AI feature and receive a persisted suggestion record.
AI writing assistant	AI-1.1	Sub-requirement	The system shall invoke AI against the user's explicit selection range rather than defaulting to the full document.	A user highlights text and clicks an AI action.	The server extracts the selected content and stores the selection metadata with the interaction.	The persisted AI interaction includes `selection.start`, `selection.end`, and the captured source text.

Continued on next page

Table 2: Functional requirements specification (Continued)

Area	ID	Level	Description	Trigger	Expected system behavior	Acceptance criteria
AI writing assistant	AI-1.2	Sub-requirement	The system shall present AI output as a proposal rather than mutating the document immediately.	An AI job completes.	The client receives a completed suggestion event and displays the proposed text in the AI panel.	The document content remains unchanged until the user explicitly applies the suggestion.
AI writing assistant	AI-1.3	Sub-requirement	The system shall let users apply or reject completed AI suggestions and record the resulting status.	A user clicks apply or reject on a completed suggestion.	The system updates the interaction status to `applied`, `partially_applied`, or `rejected`, and versions the document if content changes.	Applying a suggestion creates a new document version; rejecting it updates the interaction without changing content.
AI writing assistant	AI-1.4	Sub-requirement	The system shall show the user the exact scoped text or cursor context about to be sent to the AI before submission.	A user selects text, moves the cursor, or changes the target language before invoking AI.	The client previews the active selection or cursor context and the destination language before dispatching the request.	The UI shows bounds or cursor position plus a text preview that matches the persisted `sourceText` stored with the AI interaction.
Document management	DOC-1	Capability	The system shall manage the full document lifecycle including creation, loading, versioning, sharing, and export readiness.	A user creates, opens, shares, or revisits a document.	The system persists the document aggregate and enforces its access policy consistently.	Authorized users can create and reopen documents; unauthorized users cannot access them.
Document management	DOC-1.1	Sub-requirement	The system shall allow authorized users to create and load documents through concrete API contracts.	A user submits a new document or opens an existing one.	The API returns structured document data matching the shared schema.	`POST /v1/documents` returns a document with ID, timestamps, permissions, versions, and AI-history arrays.
Document management	DOC-1.2	Sub-requirement	The system shall maintain a version history for every persisted content mutation.	A document is edited, reverted, or updated through AI apply.	The server stores a new immutable version record with author, reason, and timestamp.	`GET /v1/documents/:id/versions` returns the ordered version history including reason metadata.

Continued on next page

Table 2: Functional requirements specification (Continued)

Area	ID	Level	Description	Trigger	Expected system behavior	Acceptance criteria
Document management	DOC-1.3	Sub-requirement	The system shall allow owners to share a document with role-specific permissions and AI-enablement flags.	An owner updates sharing settings.	The server updates or creates a permission grant for the selected principal.	A share request changes the target principal's role and AI access without affecting other grants.
Document management	DOC-1.4	Sub-requirement	The system shall support export to PDF, DOCX, and Markdown in the target architecture.	A user chooses an export format.	The system produces a point-in-time document snapshot in the requested format.	Export requests produce a downloadable artifact with the selected version and optional tracked AI changes.
User management	USER-1	Capability	The system shall authenticate users, maintain sessions, and enforce role-based authorization at the document level.	A user logs in or attempts a protected action.	The system resolves the caller identity, session, and document-specific permission before executing the request.	Protected endpoints reject missing or invalid sessions with `UNAUTHORIZED` or `FORBIDDEN`.
User management	USER-1.1	Sub-requirement	The system shall create time-bounded sessions after successful authentication.	A valid user logs in.	The system issues a session token with an expiry timestamp and binds it to the user.	`POST /v1/auth/login` returns a token, user ID, and expiry timestamp.
User management	USER-1.2	Sub-requirement	The system shall enforce the roles `owner`, `editor`, `commenter`, and `viewer` with different action permissions.	A user attempts to edit, revert, share, or invoke AI.	The policy layer checks the caller's role and AI flag before allowing the action.	Viewers cannot edit, commenters cannot mutate content, and only owners can change sharing.
User management	USER-1.3	Sub-requirement	The system shall make AI access independently configurable from edit permissions.	An owner or admin changes document AI policy.	The permission grant stores `allowAi` separately from the document role.	A commenter with `allowAi=true` can request AI suggestions while remaining unable to edit document content directly.

1.3 Non-Functional Requirements

Table 3: Non-functional requirements

ID	Attribute	Measurable requirement	Justification
NFR-LAT-1	Latency	Keystroke propagation between active collaborators shall be visible within 250 ms p95 inside the same region.	Longer delay breaks the feeling of co-presence and makes simultaneous editing feel unsafe.
NFR-LAT-2	Latency	AI response initiation shall return an accepted request or quota/error result within 1.5 s p95.	Users should know quickly whether the AI request is queued, denied, or failed.
NFR-LAT-3	Latency	Opening a document under 1 MB shall show the first interactive editor state within 2.0 s p95 on broadband.	Shared editing must feel immediate enough for drafting sessions and classroom demos.
NFR-SCALE-1	Scalability	The target architecture shall support 50 concurrent editors per document. The PoC validates contracts rather than that scale target.	Shared planning documents often involve workshops or live review sessions.
NFR-SCALE-2	Scalability	The target architecture shall support 25,000 concurrently open documents system-wide and horizontal growth of stateless web/API nodes.	Growth should come from adding nodes instead of re-architecting session flow.
NFR-COST-1	Cost efficiency	AI usage shall be bounded by configurable quotas and model-tier selection so spend can be capped per tenant, environment, or demo deployment.	Founders and operators need predictable operating cost as AI usage scales.
NFR-AVL-1	Availability	Target service availability shall be 99.9% monthly excluding planned maintenance.	The product is a collaboration tool and should be dependable during working hours.
NFR-AVL-2	Availability	During partial AI-service outage, core editing, sharing, and version history shall remain available while AI actions degrade to retryable errors.	AI is important but must not take down the editor itself.

Continued on next page

Table 3: Non-functional requirements (Continued)

ID	Attribute	Measurable requirement	Justification
NFR-OPS-1	Observability	The system shall emit structured logs and core metrics for authentication, document loads, version conflicts, active realtime sessions, export requests, and AI lifecycle events.	Realtime collaboration and async AI failures are difficult to diagnose without first-class telemetry.
NFR-SEC-1	Security and privacy	All traffic shall use TLS in transit, and document content plus AI logs shall be encrypted at rest in the production architecture.	Documents may contain sensitive project plans, legal drafts, or internal strategy.
NFR-SEC-2	Security and privacy	AI requests shall send only the minimum scoped text and prompt metadata required for the feature; default retention for raw AI logs shall not exceed 30 days.	Minimizing third-party exposure reduces compliance risk and cost.
NFR-SEC-3	Security and privacy	Secrets, API keys, and database credentials shall never be committed to source control and must be injected through environment configuration.	This is a baseline control and also essential for team collaboration.
NFR-USAB-1	Usability	When a large document has many active collaborators, the UI shall collapse detailed cursor data into a summarized presence bar after 10 visible users.	The editor must remain readable rather than overwhelmed by indicators.
NFR-USAB-2	Usability	The web client shall satisfy WCAG 2.1 AA contrast for primary text and interactive controls and remain usable by keyboard alone.	Collaboration tools are used across varied devices and accessibility needs.

1.4 User Stories and Scenarios

Table 4: User stories and expected behavior

ID	User story	Expected behavior / design choice
US-01	As an editor, I want my changes to appear for collaborators while I type so that we can draft together.	Live changes travel over the real-time session and update the shared version immediately after validation.
US-02	As an editor, I want to recover gracefully if someone else changed the same paragraph before my update landed.	The client receives `VERSION_CONFLICT`, reloads the latest snapshot, and the user can continue from the current shared state.
US-03	As a collaborator, I want to see who is online and where their cursor is so that coordination feels natural.	Presence pills show active users and their latest cursor index in the current document.
US-04	As a user who went offline mid-edit, I want the session to reconnect automatically when connectivity returns.	The client re-establishes the WebSocket session, reloads the latest document snapshot, and rejoins presence.
US-05	As a writer, I want to select a paragraph and ask the AI for a summary so that I can create an executive brief quickly.	AI requests are scoped to the selection, previewed before submission, and returned as proposals in the side panel.
US-06	As a writer, I want to translate a section into another language so that I can prepare localized versions of the draft.	The translation request includes the target language and returns a persisted suggestion for later apply or reject.
US-07	As a writer, I want to partially accept an AI-generated rewrite so that I stay in control of the final phrasing.	The target architecture supports partial merge and records `partially_applied`; the PoC exposes full apply and the supporting status model.
US-08	As a team lead, I want to revert a document to a previous version while others are editing so that I can undo a bad change quickly.	Revert creates a new top-of-history version rather than deleting history, and the resulting content is pushed to active collaborators.
US-09	As a document owner, I want to share a document with read-only access so that stakeholders can review without changing content.	Sharing creates or updates a permission grant with role `viewer`.
US-10	As a team lead, I want to review AI interaction history so that I understand how a section evolved.	AI requests and their statuses remain attached to the document for later inspection.

Continued on next page

Table 4: User stories and expected behavior (Continued)

ID	User story	Expected behavior / design choice
US-11	As a commenter, I want to invoke AI without editing the raw document so that I can propose wording changes safely.	AI access is separated from edit access through the `allowAi` flag.
US-12	As a viewer, I want the UI to stop me from editing gracefully so that I understand my role without trial-and-error.	The editor becomes read-only and the UI explains why content is locked.

1.5 Requirements Traceability Matrix

Table 5: Traceability matrix from user stories to requirements and architecture

User story	Functional requirements	Architecture components
US-01	COLL-1, COLL-1.1	Web editor, realtime gateway, document service
US-02	COLL-1.3, DOC-1.2	Web editor, realtime gateway, document service, document store
US-03	COLL-1.2	Presence UI, realtime gateway
US-04	COLL-1, USER-1.1	Session manager, realtime gateway, auth middleware
US-05	AI-1, AI-1.1, AI-1.2, AI-1.4	AI panel, application API, AI orchestrator
US-06	AI-1, AI-1.1	AI panel, application API, AI orchestrator
US-07	AI-1.3, DOC-1.2	AI panel, document service, document store
US-08	DOC-1.2, COLL-1.1	Version-history UI, application API, document service, realtime gateway
US-09	DOC-1.3, USER-1.2	Sharing UI, application API, permission policy
US-10	AI-1.3, DOC-1.2	AI history panel, document store, application API
US-11	USER-1.3, AI-1	Permission policy, AI orchestrator, AI panel
US-12	USER-1.2	Web editor, permission policy

2 System Architecture

2.1 Architectural Drivers

Table 6: Ranked architectural drivers

Rank	Driver	Why it dominates the design
1	Low-latency collaborative editing	The core product fails if shared edits feel slow or unsafe. This drove the push-based real-time channel and explicit version-conflict handling.
2	Security and privacy of document content	Sensitive text plus third-party LLM processing requires scoped context, role checks, and audit-friendly data models.
3	Predictable AI user experience	AI is a product feature rather than a novelty. Suggestions must be proposals, asynchronous, role-aware, and undoable through version history.
4	Modularity for team parallelism	Frontend, backend, contracts, and AI flows need clean boundaries so multiple developers can work safely in parallel.
5	Auditability and recovery	Version history and AI interaction records are essential because collaborative writing regularly needs rollback and explanation.

If the ranking changed, the architecture would change as well. For example, if cost minimization outranked collaboration latency, the system would favor heavier polling, reduced presence detail, and more batched AI flows. Instead, this design spends complexity on WebSocket propagation and explicit proposal workflows because user trust in collaboration is the primary architectural driver.

2.2 System Design Using the C4 Model

2.2.1 Level 1: System Context Diagram

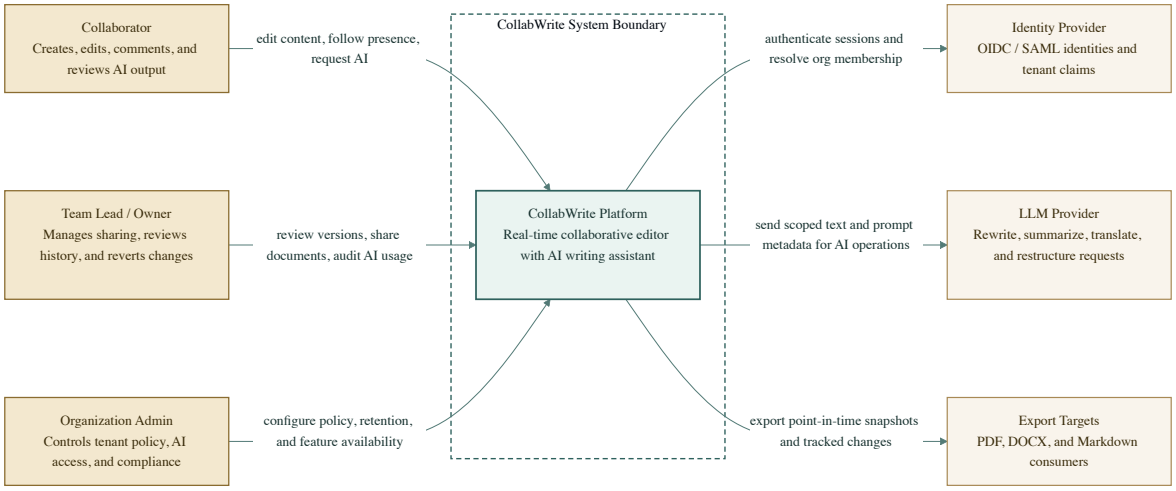

Figure 1: Level 1 system context diagram for CollabWrite.

The system context places CollabWrite between three human actor categories and three external systems. The most important external boundary is the LLM provider because it intro-

duces privacy, latency, and cost trade-offs. Identity remains external as well so authentication can evolve without changing document-domain logic.

2.2.2 Level 2: Container Diagram

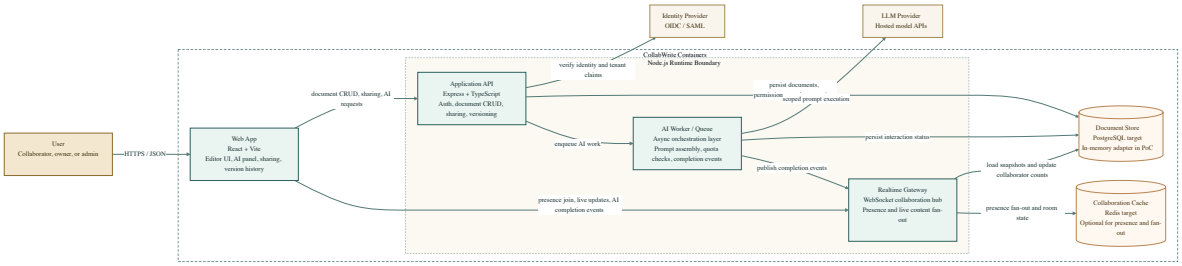

Figure 2: Level 2 container view for CollabWrite.

The container design separates the web client from the application API and the realtime collaboration gateway. In the PoC, the REST API and WebSocket gateway run in the same Node.js process, but they remain distinct containers logically because they solve different problems and will scale differently later. The implementation now also includes a dedicated FastAPI export worker so binary document rendering stays isolated from the interactive Node.js collaboration path. The target architecture also introduces an AI worker / queue boundary so long-running model calls do not block interactive API behavior.

2.2.3 Level 3: Component Diagram for the Application API

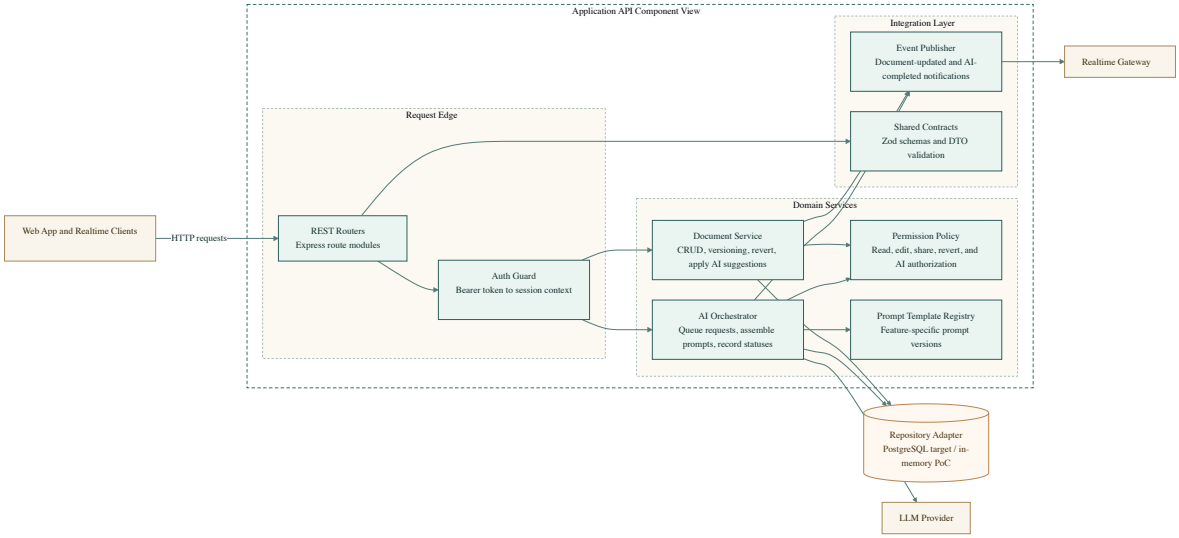

Figure 3: Level 3 component view for the application API container.

The application API container is the best Level 3 candidate because it contains most of the business rules: authorization, versioning, AI orchestration, and contract validation. The imple-

mented PoC mirrors this decomposition through route modules, services, a collaboration hub, and an in-memory repository adapter.

Full Mermaid source listings for Figures 1 to 3, plus the data model and AI lifecycle diagrams, are provided in Appendix A.

2.2.4 Feature Decomposition

Table 7: Feature decomposition and module contracts

Module	Responsibility	Depends on	Exposes
Web editor module	Owns text-editing UX, selection capture, document switching, and read-only behavior by role	Shared contracts, REST API, realtime gateway	Document view state, selection range, editor events
Frontend collaboration state	Tracks current document, current version, participants, reconnect state, and AI-panel state	WebSocket channel, shared message schemas	Presence UI, reconnect logic, socket message handlers
Realtime synchronization layer	Validates join context, tracks presence, applies live edits, and broadcasts updates	Auth service, document service, shared contracts	WebSocket session `/ws`, `presence.*`, `document.updated`, and `ai.completed` messages
AI assistant service	Persists AI interactions, scopes context, schedules async completion, enforces quota and policy, and selects the live or fallback provider	Document service, repository, OpenAI Responses adapter, or mock adapter	AI request creation, completion lifecycle, provider status, and suggestion updates
Document storage and versioning	Stores document aggregate, immutable versions, permissions, and AI history	Repository adapter	CRUD, version history, revert, share, and apply-suggestion operations
Export rendering service	Produces PDF, DOCX, and Markdown artifacts from point-in-time document snapshots and optional AI history	Application API, Python document libraries	`POST /render` and binary export payloads
Authentication and authorization	Resolves session tokens and enforces role rules plus `allowAi`	Session store, permission grants	Auth middleware, login endpoint, access decisions

Continued on next page

Table 7: Feature decomposition and module contracts (Continued)

Module	Responsibility	Depends on	Exposes
Shared contract package	Maintains request/response/message schemas for web and API	Zod and TypeScript	`@midterm/shared` DTOs and runtime parsers

2.3 AI Integration Design

2.3.1 Context and Scope

AI sees the explicit user selection by default, and the current implementation also passes a narrow surrounding context window for higher-quality rewrites plus a cursor-local context window for inline continuation. This minimizes privacy risk, reduces token cost, and improves prompt relevance without sending the full document body to the provider by default. For very long documents, the system still loads only the requested segment plus lightweight local context instead of the full draft.

2.3.2 Trustworthiness of Cloud-Based Completion

Inline completion is the most trust-sensitive AI feature in the PoC because it feels like local autocomplete while still depending on a cloud model. The design therefore uses a deliberately conservative contract:

1. completion is explicit and user-triggered through the `Continue Writing` action rather than being sent on every keystroke;
2. the provider receives only the document title plus bounded text immediately before and after the cursor, not the full draft by default;
3. the UI exposes whether the system is using the live OpenAI provider or the fallback mock path;
4. the returned text is treated as a proposal, not as silent ghost text, so the writer keeps editorial control;
5. each interaction stores the initiator, model, prompt-template version, and requested document version for later inspection; and
6. completion apply requests are rejected if the document changed in the meantime.

This means the completion feature is cloud-based but still explainable in terms of bounded disclosure, explicit invocation, inspectable provenance, and safe application semantics. A production always-on autocomplete experience would require stronger tenant policy controls, retention guarantees, and provider evaluation before it should be trusted with high-sensitivity content.

2.3.3 Suggestion UX

AI output is presented as a tracked proposal in a side panel rather than as an immediate inline mutation. This design:

1. keeps authors in control,
2. makes AI history inspectable, and
3. supports later partial-merge and tracked-change export workflows.

The current PoC supports apply and reject. The target product also supports partial acceptance, which is already represented in the data model through the `partially_applied` state.

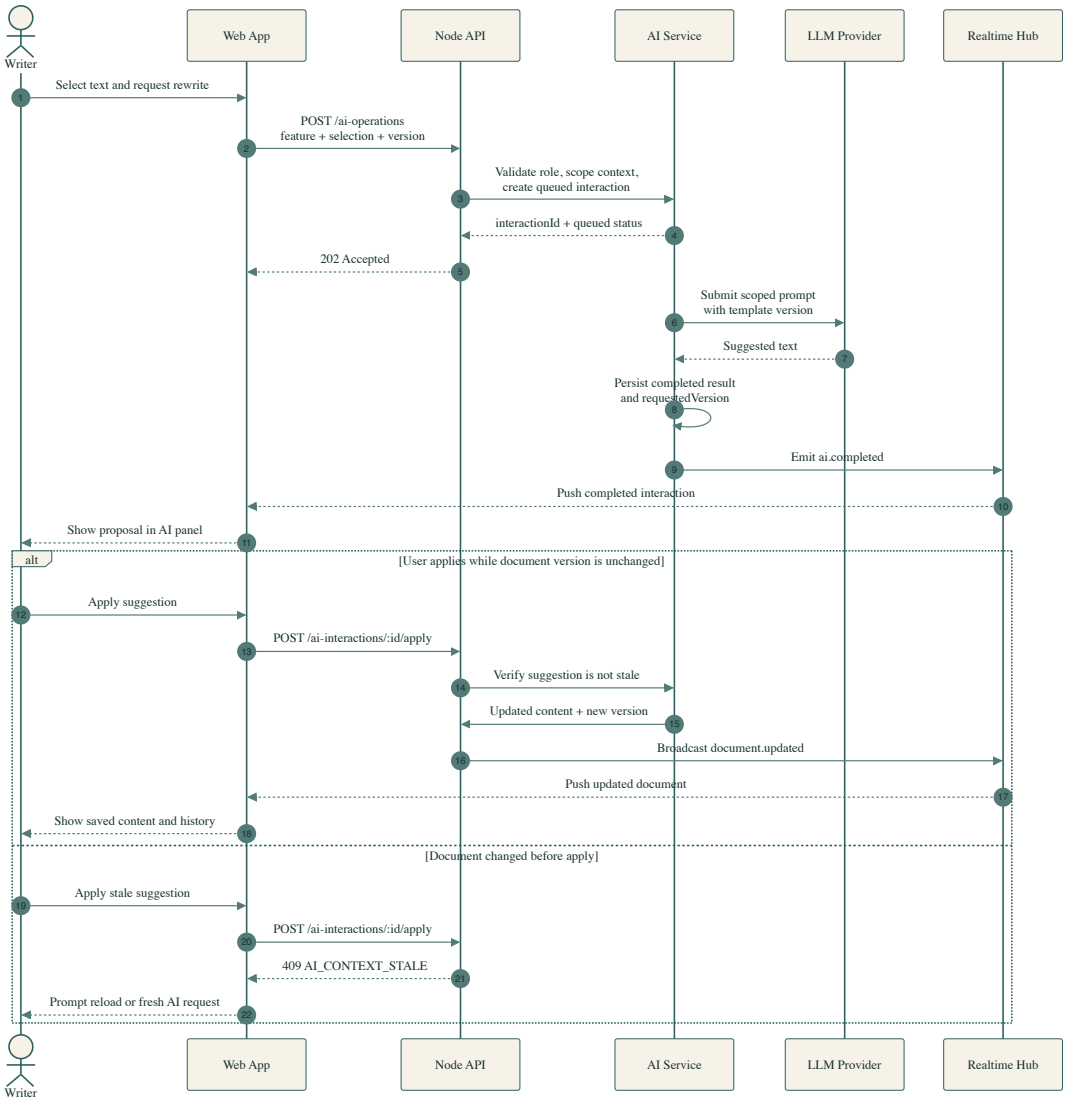

Figure 4: Asynchronous AI suggestion lifecycle in the implemented PoC.

Figure 4 makes one of the most important design choices explicit: AI is handled as an asynchronous proposal workflow rather than a synchronous content mutation. The client gets a fast acceptance response, the server performs prompt assembly and provider interaction out of band, and the final suggestion re-enters the user experience through the same realtime channel used for collaborative document updates.

2.3.4 AI During Collaboration

The system does not lock the selected region during AI generation. Other edits may continue. Because of that, completed suggestions are version-scoped: if the document changes after the request was created, the server rejects a blind apply attempt and forces the client to reload or request a fresh suggestion. If the user applies a still-valid suggestion, that application becomes a new version and is broadcast to the session like any other edit. This avoids pessimistic-locking complexity while preserving recoverability through version history.

2.3.5 Prompt Design

Prompt logic is template-based and versioned rather than hardcoded inside controllers. Each AI interaction records `promptTemplateVersion` so the team can compare behavior over time and revise prompts without losing traceability. The implemented provider layer also forces structured output parsing so the model returns only the insertion or replacement text expected by the editor.

2.3.6 Model and Cost Strategy

The implemented AI layer uses different model classes by task:

- rewrite and restructure use a higher-quality model;
- summarize and translation use cheaper, faster models where acceptable;
- quota management combines per-user daily limits with tenant budget caps.

When quota is exceeded, the API returns a specific `quota_exceeded` state so the client can distinguish this from a transient model failure. When the live provider is not configured, the same orchestration path falls back to a deterministic mock adapter so local development remains runnable.

2.4 API Design

2.4.1 API Style

The communication model deliberately mixes synchronous REST with asynchronous push:

- REST handles CRUD, sharing, version history, login, and AI request creation because these actions are explicit, auditable, and easy to test.
- REST also brokers export requests because export is an explicit document-management operation that returns a point-in-time artifact.
- WebSocket handles presence and live content propagation because polling would degrade collaboration UX and waste resources.

2.4.2 REST API Contract

Table 8: REST API contract

Interaction	Method and path	Request contract	Response contract	Notes
Login	`POST /v1/auth/login`	`{ userId }`	`{ session, user }`	PoC uses seeded users; production swaps to IdP-backed login
Session lookup	`GET /v1/auth/me`	Bearer token	`{ session, user }`	Validates token and refreshes client context
List users	`GET /v1/users`	none	`{ users[] }`	Used for demo login and sharing UI
AI provider status	`GET /v1/ai/provider`	Bearer token	`{ provider }`	Exposes whether the app is using live OpenAI or the fallback adapter
List documents	`GET /v1/documents`	Bearer token	`{ documents[] }`	Returns only documents visible to the caller
Create document	`POST /v1/documents`	`{ title, content }`	`{ document }`	Seeds initial owner permission and version 1
Load document	`GET /v1/documents/:id`	Bearer token	`{ document }`	Returns content, permissions, versions, and AI history
Update content	`PUT /v1/documents/:id/content`	`{ baseVersion, content, reason }`	`{ document, savedVersion }`	Rejects stale base version
List versions	`GET /v1/documents/:id/versions`	Bearer token	`{ versions[] }`	Ordered newest-first
Revert version	`POST /v1/documents/:id/revert`	`{ versionNumber }`	`{ document, savedVersion }`	Creates a new top-of-history version
List permissions	`GET /v1/documents/:id/permissions`	Bearer token	`{ permissions[] }`	Used to inspect access state
Share document	`POST /v1/documents/:id/share`	`{ principalType, principalId, role, allowAi }`	`{ permissions[] }`	Owner-only operation
Export document	`POST /v1/documents/:id/export`	`{ format, includeAiAppendix }`	binary attachment	Proxies a snapshot to the FastAPI renderer and returns PDF, DOCX, or Markdown
List AI interactions	`GET /v1/documents/:id/ai-interactions`	Bearer token	`{ interactions[] }`	Supports reload and auditing
Request AI	`POST /v1/documents/:id/ai-operations`	`{ feature, selection, targetLanguage? }`	`{ interaction }`	Returns `202 Accepted` or `429` quota result
Apply AI suggestion	`POST /v1/documents/:id/ai-interactions/:iid/apply`	`{ mode, acceptedText? }`	`{ document }`	Creates a new version if content changes and rejects stale AI context

Continued on next page

Table 8: REST API contract (Continued)

Interaction	Method and path	Request contract	Response contract	Notes
Reject AI suggestion	`POST /v1/documents/:id/ai-interactions/:iid/reject`	`{}`	`{ interactions[] }`	Marks the suggestion as rejected

2.4.3 Realtime Session Contract

The client joins a document session through `ws://host/ws?token=<session>&documentId=<documentId>`. Server-to-client messages include `presence.snapshot`, `presence.updated`, `document.updated` and `ai.completed`. Client-to-server messages include `document.change` and `cursor.change`.

2.4.4 Long-Running AI Operations

From the client perspective, AI is asynchronous:

1. the client submits `POST /ai-operations`;
2. the API immediately returns a queued interaction record;
3. the AI worker completes later and emits `ai.completed` over WebSocket;
4. the client updates the side panel and waits for explicit apply or reject.

This avoids long synchronous HTTP waits and distinguishes queue acceptance from model completion.

2.4.5 Error Handling Strategy

Clients distinguish failure modes by explicit error codes or interaction state:

- AI slow: request accepted but interaction remains `queued` or `in_progress`;
- AI failed: interaction becomes `failed`;
- quota exceeded: HTTP `429` with interaction state `quota_exceeded`;
- stale AI suggestion: HTTP `409` with `AI_CONTEXT_STALE`;
- stale edit: HTTP or socket error `VERSION_CONFLICT`;
- unauthorized: `UNAUTHORIZED`;
- forbidden action: `FORBIDDEN`, `EDIT_NOT_ALLOWED`, `SHARE_NOT_ALLOWED`, or `AI_NOT_ALLOWED`.

2.5 Authentication and Authorization

Authentication is required because the product manages private content and document-scoped permissions. The system expects regular collaborators, document owners or team leads, and organization administrators.

Table 9: Document roles and permissions

Role	Read	Edit	Revert	Share	Invoke AI
Owner	Yes	Yes	Yes	Yes	Yes

Continued on next page

Table 9: Document roles and permissions (Continued)

Role	Read	Edit	Revert	Share	Invoke AI
Editor	Yes	Yes	Yes	No	If `allowAi=true`
Commenter	Yes	No	No	No	If `allowAi=true`
Viewer	Yes	No	No	No	No by default

The most important privacy consideration is third-party AI processing. The design mitigates this by sending only scoped selections, logging prompt-template versions, and treating AI access as a separate policy knob rather than as a side effect of edit permission.

2.6 Communication Model

The communication model is hybrid and push-based:

- initial open: the client authenticates, fetches the document bundle over REST, and then joins the WebSocket room;
- active editing: presence and edits propagate by push to keep latency low;
- connectivity loss: the client reconnects automatically and reloads the latest snapshot before resuming edits.

This model is more complex than polling, but it aligns with the top-ranked architectural driver: low-latency collaboration.

2.7 Code Structure and Repository Organization

2.7.1 Monorepo vs. Multi-Repo

The project uses a monorepo. With a small team and heavy shared-DTO usage, a monorepo is the lowest-friction option because it keeps web, API, and shared contracts versioned together. A multi-repo split would create unnecessary coordination cost every time request or response schemas evolve.

2.7.2 Directory Layout

Listing 1: Repository layout

```
.
|-- apps
|   |-- api
|   |   `-- src
|   |       |-- middleware
|   |       |-- realtime
|   |       |-- repositories
|   |       |-- routes
|   |       |-- services
|   |       |-- utils
|   |       |-- app.ts
|   |       |-- app.test.ts
```

```
|   |       `-- server.ts
|   `-- web
|       |-- index.html
|       `-- src
|           |-- components
|           |-- styles
|           |-- api.ts
|           |-- App.tsx
|           `-- main.tsx
|-- packages
|   `-- shared
|       `-- src
|           |-- index.ts
|           `-- index.test.ts
|-- services
|   `-- export-api
|       |-- main.py
|       |-- requirements.txt
|       `-- test_main.py
|-- docs
|   |-- diagrams
|   `-- report
|-- package.json
`-- README.md
```

2.7.3 Shared Code

Shared request, response, and realtime message schemas live in `packages/shared`. This package exposes Zod schemas plus inferred TypeScript types. The API uses them for runtime validation; the frontend uses them for compile-time safety and message handling.

2.7.4 Configuration Management

Secrets are environment-driven. The PoC needs variables such as `VITE_API_URL` and `EXPORT_SERVICE_URL`, while the production architecture expects LLM keys, database URLs, and session secrets to live in environment variables or a secret manager. `.env` files are ignored and `.env.example` documents the required keys.

2.7.5 Testing Structure

Tests sit next to source where practical:

- `packages/shared/src/index.test.ts` for contract tests;
- `apps/api/src/app.test.ts` for API integration tests;
- `services/export-api/test_main.py` for FastAPI export-worker tests.

Planned test layers include unit tests for prompt selection and permission policy, integration tests for REST contracts and repository behavior, and end-to-end tests for multi-user collaboration plus AI suggestion UX. AI integration is tested through deterministic provider stubs plus a mock-provider path so local and CI runs do not require paid API calls. Export integration is split cleanly: the Node.js API tests the proxy contract, while the Python worker tests binary rendering behavior directly.

2.8 Data Model

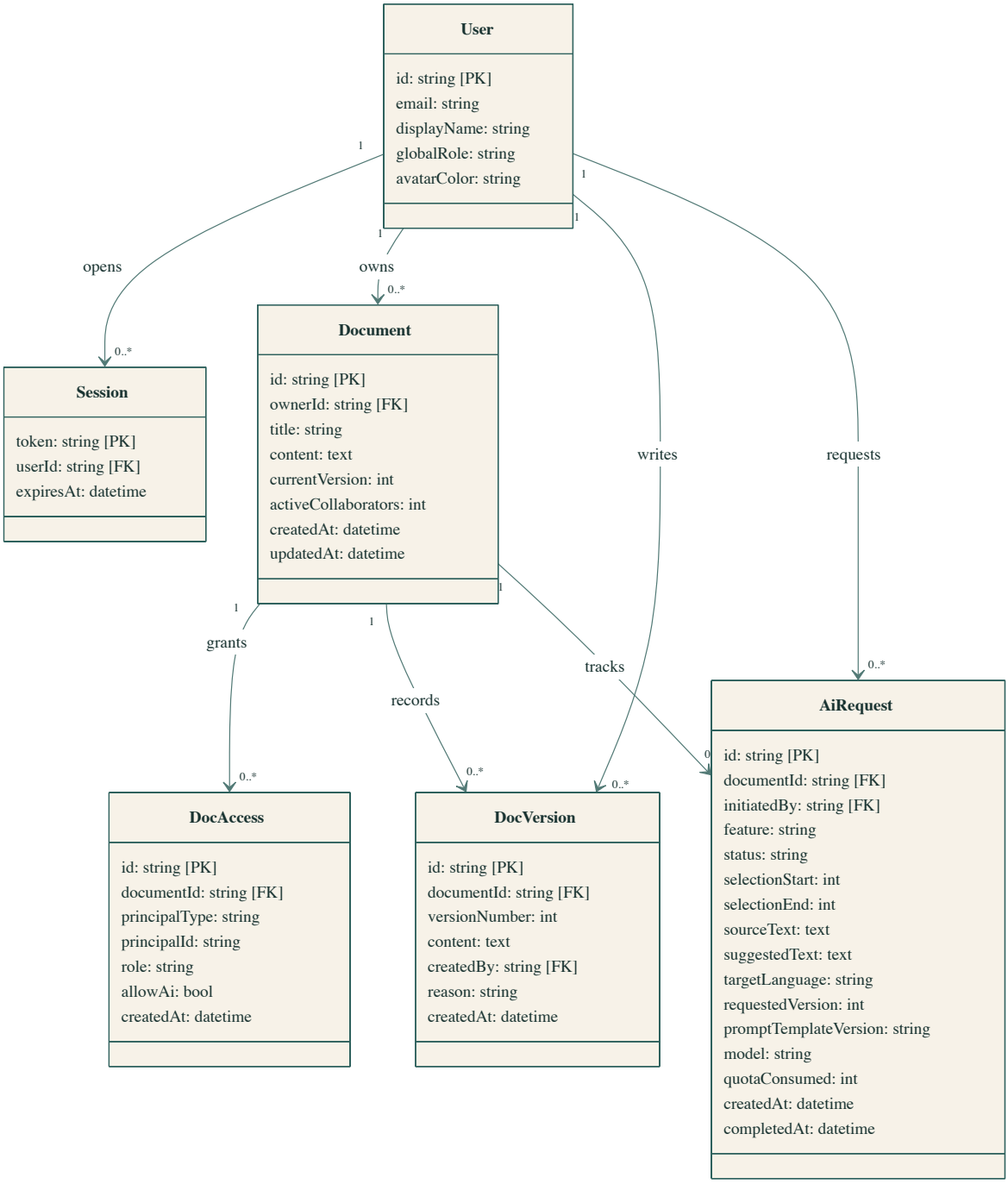

Figure 5: Logical data model diagram for the implemented PoC.

The document aggregate stores more than content: ownership, current version, timestamps, permissions, immutable versions, and AI interaction history. This is necessary because the

product needs accountability, not just raw text storage. Three modeling decisions are especially consequential:

- version history is immutable, so revert creates a new version rather than deleting old ones;
- AI interactions keep both source and suggested text so reviewers can compare and audit;
- permissions support users, teams, and link sharing even though the PoC currently implements user principals only.

2.9 Architecture Decision Records

2.9.1 ADR-01: Monorepo with Shared Contract Package

- **Status:** Accepted
- **Context:** Web, API, and realtime flows evolve together. Schema drift would slow the team and break demos.
- **Decision:** Keep frontend, backend, and shared contracts in one repository and publish DTOs through an internal workspace package.
- **Consequences:** Faster cross-layer changes, easier traceability, and a single CI pipeline. The trade-off is a larger repository and less independent deployment freedom.
- **Alternatives considered:** Separate repos for web and backend were rejected because every API change would require cross-repo synchronization and duplicate type definitions.

2.9.2 ADR-02: REST for CRUD plus WebSocket for Collaboration

- **Status:** Accepted
- **Context:** The system needs both strongly testable CRUD contracts and low-latency live editing.
- **Decision:** Use REST for explicit resource operations and WebSocket for presence plus content propagation.
- **Consequences:** Better UX and clearer boundaries, at the cost of maintaining two communication paths. That complexity is justified by the top-ranked latency driver.
- **Alternatives considered:** Polling was rejected because it increases latency and server load. An all-WebSocket design was rejected because it complicates auditable CRUD and testing.

2.9.3 ADR-03: AI Suggestions Are Proposals, Not Immediate Mutations

- **Status:** Accepted
- **Context:** Direct AI mutation is fast to implement but reduces user trust and complicates collaboration recovery.
- **Decision:** AI writes suggestion records that users explicitly apply or reject.

- **Consequences:** Safer collaboration, easier auditing, and clearer version history. The trade-off is one extra interaction step for the user.
- **Alternatives considered:** Instant inline replacement was rejected because it hides AI provenance and increases accidental destructive edits.

2.9.4 ADR-04: Repository Abstraction with In-Memory PoC Adapter

- **Status:** Accepted
- **Context:** The assignment requires a running PoC now, but the semester project will outgrow in-memory state quickly.
- **Decision:** Implement the PoC against an in-memory repository that already matches the future service and aggregate boundaries.
- **Consequences:** The demo remains lightweight while preserving a clean migration path to PostgreSQL and Redis later. The trade-off is that the PoC does not validate production persistence or multi-instance coordination yet.
- **Alternatives considered:** Hardwiring state directly inside route handlers was rejected because it would not scale architecturally or align with the report.

3 Project Management and Team Collaboration

3.1 Team Structure and Ownership

Table 10: Team structure and ownership model

Team member role	Ownership area	Responsibilities
Ahmed Aymna — Backend / integration lead	`apps/api`, shared contracts, final integration	REST contracts, auth, sharing, versioning, export integration, cross-module merge resolution, and submission readiness
Aya — Frontend / UX lead	`apps/web`, demo UX, interaction design	Editor UX, presence visualization, sharing and export affordances, AI-panel interaction flow, and demo polish
Mohamed Ibrahim — Requirements / architecture lead	Requirements engineering, architecture narrative, report traceability	Stakeholder analysis, functional and non-functional requirements, user stories, traceability, and report consistency across sections
Praveen Anilkumar Shukla — Realtime / AI / platform lead	`apps/api/realtime`, AI orchestration, environment setup	Realtime session behavior, observability, AI workflow coordination, provider integration boundaries, and runtime environment setup

Features that span ownership, such as AI-assisted editing, are delivered through one cross-functional issue with explicit subtasks per owner and one designated integrator. Ahmed Aymna acts as the final integrator for submission branches, while Mohamed Ibrahim owns consistency between implemented behavior and the written report. Technical disagreements follow a lightweight ADR or RFC process; unresolved issues escalate to Ahmed Aymna plus one additional cross-area reviewer within 24 hours.

3.2 Development Workflow

- **Branching strategy:** short-lived feature branches from `main`, named `feat/<scope>-<ticket>`, `fix/<scope>-<ticket>`, or `docs/<scope>-<ticket>`.
- **Merge policy:** pull requests only; no direct pushes to `main`; squash merge after review.
- **Code review:** at least one approving review from the owning area and one cross-area reviewer for shared contracts or cross-cutting features.
- **Approval criteria:** passing tests, no schema drift, updated docs for new endpoints, and clear rollback behavior for risky changes.
- **Issue tracking:** backlog items in GitHub Projects or Jira with labels such as `web`, `api`, `ai`, `realtime`, and `infra`.
- **Communication:** Slack or Microsoft Teams for daily coordination; durable decisions are copied into ADRs or issue comments so they do not disappear in chat history.

3.3 Development Methodology

The team uses a lightweight Scrum-style methodology with one-week iterations from **April 5, 2026** through **May 24, 2026**. Work is prioritized by architectural dependency first and user-visible value second:

1. shared contracts and repository boundaries,
2. document CRUD and auth,
3. realtime editing and presence,
4. AI workflow and quota handling,
5. export and hardening.

Infrastructure, testing, and data-model work are tracked as first-class backlog items with acceptance criteria, not treated as side work.

3.4 Risk Assessment

Table 11: Project-specific risk register

Risk	Likelihood	Impact	Mitigation	Contingency
Real-time edit conflicts become too frequent with naive full-document updates	Medium	Users lose trust in collaboration and edits may be overwritten	Move from full-document pushes to OT/CRDT or patch-based operations; add conflict telemetry early	Freeze simultaneous same-region editing and fall back to optimistic locking plus explicit conflict-resolution UI
Third-party AI latency or outages degrade the whole product	High	AI feels broken, but editing must continue	Keep AI asynchronous, decouple through a queue, and surface retryable states	Disable AI features temporarily while preserving editing, sharing, and history
AI cost grows unpredictably with large selections and repeated requests	Medium	Budget overrun or forced throttling	Enforce scoped context, quotas, tenant budget caps, and prompt templates by feature	Reduce available AI features or switch to lower-cost models for non-critical tasks
Schema drift between frontend and backend breaks integration late in the sprint	Medium	Demo failures and lost integration time	Keep shared contracts in a workspace package and test responses against schemas	Temporarily freeze feature work and repair contract mismatches before resuming
Ownership boundaries fail and cross-cutting features merge poorly	Medium	Integration stalls, PRs become large, and bug rates increase	Use clear module ownership, small PRs, and an explicit integrator for cross-functional work	Re-scope the sprint, assign one engineer to integration-only work, and defer lower-priority features

3.5 Timeline and Milestones

Table 12: Semester timeline and milestones

Date	Milestone	Acceptance criteria
April 7, 2026	Contract baseline complete	Shared DTO package exists; login and document CRUD schemas are agreed and reviewed.
April 14, 2026	Authenticated CRUD slice complete	API serves document create/load/update with auth; integration tests pass.
April 21, 2026	Realtime collaboration slice complete	Two clients can join the same document, see presence, and receive shared edits under target demo conditions.
April 28, 2026	Versioning and sharing complete	Version history and owner-managed permission updates work through the UI and API.
May 8, 2026	AI proposal workflow complete	AI requests queue asynchronously, suggestions appear in the UI, and apply/reject flows persist status correctly.
May 17, 2026	Hardening sprint complete	README, diagrams, ADRs, risk register, tests, and environment setup are current; demo script is rehearsed.
May 24, 2026	Final submission candidate	End-to-end demo passes from a fresh clone, the report is exported to PDF, and repository state is submission-ready.

4 Proof of Concept

4.1 What the PoC Demonstrates

The implemented PoC validates the architectural skeleton directly in code:

- a working React frontend in `apps/web`;
- a working Express API in `apps/api`;
- shared Zod contracts in `packages/shared`;
- a dedicated FastAPI export worker in `services/export-api`;
- authenticated document listing and loading;
- document creation, sharing, version history, and revert;
- role-aware AI request, proposal, apply, and reject flow;
- live OpenAI-backed rewrite, summarize, translate, restructure, proofread, and inline-completion workflows with a mock fallback path;
- export of point-in-time document snapshots to PDF, DOCX, and Markdown with an optional AI-history appendix;
- WebSocket presence and live content propagation.

4.2 What the PoC Intentionally Does Not Implement Yet

- production identity-provider integration;
- persistent PostgreSQL or Redis infrastructure;
- OT/CRDT-grade merge logic for same-character concurrent edits;
- tenant billing, provider-routing policy, and enterprise governance controls around AI usage.

These omissions are deliberate. The PoC proves the contracts, module boundaries, and communication model without pretending the prototype is already production-ready.

4.3 Architecture-to-Code Alignment

Table 13: Architecture-to-code alignment

Architecture element	PoC implementation
Web app container	`apps/web` React client
Application API container	`apps/api` Express app and route modules
Realtime gateway	`apps/api/src/realtime/collaboration-hub.ts`

Continued on next page

Table 13: Architecture-to-code alignment (Continued)

Architecture element	PoC implementation
Export rendering worker	`services/export-api/main.py` FastAPI service for PDF, DOCX, and Markdown
Shared contracts	`packages/shared/src/index.ts`
Document store	`apps/api/src/repositories/in-memory-store.ts` as the PoC adapter
AI orchestrator	`apps/api/src/services/ai-service.ts` with async suggestion completion

4.4 How to Run the PoC

Use the root-level commands:

Listing 2: PoC setup and run commands

```
npm install
python3 -m venv .venv-export
source .venv-export/bin/activate
python3 -m pip install -r services/export-api/requirements.txt
npm run dev
npm run dev:export
```

Frontend: `http://localhost:5173`
Backend: `http://localhost:4000`
Export worker: `http://127.0.0.1:8000`

4.5 Verification Evidence

The repository passes the following verification commands:

- `npm run typecheck`
- `npm test`
- `source .venv-export/bin/activate && npm run test:export`
- `npm run build`
- `npm run render:report`

The backend was also smoke-tested through live `curl` requests against `/health`, `POST /v1/auth/login`, `GET /v1/documents`, the authenticated AI-provider status route, and the FastAPI export-renderer health endpoint. The PoC therefore demonstrates a working end-to-end slice rather than a disconnected UI mockup.

A Mermaid Source Listings

This appendix contains the Mermaid source for each rendered architecture diagram included in the main report. The sources remain editable and versionable in the repository.

A.1 System Context Diagram

Listing 3: Mermaid source for the system context diagram

```
%%{init: {
  "theme": "base",
  "themeVariables": {
    "fontFamily": "Helvetica, Arial, sans-serif",
    "primaryColor": "#F7F2E8",
    "primaryTextColor": "#183230",
    "primaryBorderColor": "#2E615D",
    "lineColor": "#527A76",
    "secondaryColor": "#EDF4F2",
    "tertiaryColor": "#FFF8F0",
    "background": "#FFFFFF"
  },
  "flowchart": {
    "curve": "basis",
    "nodeSpacing": 38,
    "rankSpacing": 58
  }
}}%%
flowchart LR
    collaborator["Collaborator<br/>Creates, edits, comments, and reviews AI output"]:::actor
    teamlead["Team Lead / Owner<br/>Manages sharing, reviews history, and reverts changes"]:::actor
    orgadmin["Organization Admin<br/>Controls tenant policy, AI access, and compliance"]:::actor

    idp["Identity Provider<br/>OIDC / SAML identities and tenant claims"]:::external
    llm["LLM Provider<br/>Rewrite, summarize, translate, and restructure requests"]:::external
    export["Export Targets<br/>PDF, DOCX, and Markdown consumers"]:::external

    subgraph boundary["CollabWrite System Boundary"]
        direction TB
        collabwrite["CollabWrite Platform<br/>Real-time collaborative editor with AI writing assistant
     "]:::system
    end

    collaborator -->|"edit content, follow presence, request AI"| collabwrite
    teamlead -->|"review versions, share documents, audit AI usage"| collabwrite
    orgadmin -->|"configure policy, retention, and feature availability"| collabwrite

    collabwrite -->|"authenticate sessions and resolve org membership"| idp
    collabwrite -->|"send scoped text and prompt metadata for AI operations"| llm
    collabwrite -->|"export point-in-time snapshots and tracked changes"| export

    classDef actor fill:#F3E8CF,stroke:#8A6C30,stroke-width:1.5px,color:#3B2D0C;
    classDef system fill:#EAF4F1,stroke:#2E615D,stroke-width:2px,color:#183230;
    classDef external fill:#F9F4EC,stroke:#A88955,stroke-width:1.4px,color:#3B2D0C;
    style boundary fill:#FFFFFF,stroke:#2E615D,stroke-width:1.6px,stroke-dasharray: 6 4;
```

A.2 Container Diagram

Listing 4: Mermaid source for the container diagram

```
%%{init: {
  "theme": "base",
  "themeVariables": {
    "fontFamily": "Helvetica, Arial, sans-serif",
```

```
    "primaryColor": "#F7F2E8",
    "primaryTextColor": "#183230",
    "primaryBorderColor": "#2E615D",
    "lineColor": "#527A76",
    "secondaryColor": "#EDF4F2",
    "tertiaryColor": "#FFF8F0",
    "background": "#FFFFFF"
  },
  "flowchart": {
    "curve": "basis",
    "nodeSpacing": 38,
    "rankSpacing": 58
  }
}}%%
flowchart LR
    user["User<br/>Collaborator, owner, or admin"]:::actor
    idp["Identity Provider<br/>OIDC / SAML"]:::external
    llm["LLM Provider<br/>Hosted model APIs"]:::external

    subgraph system["CollabWrite Containers"]
        direction LR

        web["Web App<br/>React + Vite<br/>Editor UI, AI panel, sharing, version history"]:::container

        subgraph runtime["Node.js Runtime Boundary"]
            direction TB
            api["Application API<br/>Express + TypeScript<br/>Auth, document CRUD, sharing, versioning"]:::container
            realtime["Realtime Gateway<br/>WebSocket collaboration hub<br/>Presence and live content fan-out"]:::container
            aiworker["AI Worker / Queue<br/>Async orchestration layer<br/>Prompt assembly, quota checks, completion events"]:::container
        end

        db[("Document Store<br/>PostgreSQL target<br/>In-memory adapter in PoC")]:::storage
        cache[("Collaboration Cache<br/>Redis target<br/>Optional for presence and fan-out")]:::storage
    end

    user -->|"HTTPS / JSON"| web
    web -->|"document CRUD, sharing, AI requests"| api
    web -->|"presence join, live updates, AI completion events"| realtime

    api -->|"verify identity and tenant claims"| idp
    api -->|"persist documents, permissions, versions, sessions"| db
    realtime -->|"load snapshots and update collaborator counts"| db
    realtime -->|"presence fan-out and room state"| cache
    api -->|"enqueue AI work"| aiworker
    aiworker -->|"scoped prompt execution"| llm
    aiworker -->|"persist interaction status"| db
    aiworker -->|"publish completion events"| realtime

    classDef actor fill:#F3E8CF,stroke:#8A6C30,stroke-width:1.5px,color:#3B2D0C;
    classDef container fill:#EAF4F1,stroke:#2E615D,stroke-width:1.8px,color:#183230;
    classDef external fill:#F9F4EC,stroke:#A88955,stroke-width:1.4px,color:#3B2D0C;
    classDef storage fill:#FFF8F0,stroke:#B78452,stroke-width:1.6px,color:#4B3212;
    style system fill:#FFFFFF,stroke:#2E615D,stroke-width:1.6px,stroke-dasharray: 6 4;
    style runtime fill:#FCF8F2,stroke:#6F938F,stroke-width:1.2px,stroke-dasharray: 4 3;
```

A.3 Backend Component Diagram

Listing 5: Mermaid source for the backend component diagram

```
%%{init: {
  "theme": "base",
  "themeVariables": {
    "fontFamily": "Helvetica, Arial, sans-serif",
    "primaryColor": "#F7F2E8",
```

```
    "primaryTextColor": "#183230",
    "primaryBorderColor": "#2E615D",
    "lineColor": "#527A76",
    "secondaryColor": "#EDF4F2",
    "tertiaryColor": "#FFF8F0",
    "background": "#FFFFFF"
  },
  "flowchart": {
    "curve": "basis",
    "nodeSpacing": 35,
    "rankSpacing": 54
  }
}}%%
flowchart LR
    web["Web App and Realtime Clients"]:::external
    db[("Repository Adapter<br/>PostgreSQL target / in-memory PoC")]:::storage
    llm["LLM Provider"]:::external
    hub["Realtime Gateway"]:::external

    subgraph api["Application API Component View"]
        direction LR

        subgraph edge["Request Edge"]
            direction TB
            routes["REST Routers<br/>Express route modules"]:::component
            auth["Auth Guard<br/>Bearer token to session context"]:::component
        end

        subgraph domain["Domain Services"]
            direction TB
            docs["Document Service<br/>CRUD, versioning, revert, apply AI suggestions"]:::component
            policy["Permission Policy<br/>Read, edit, share, revert, and AI authorization"]:::component
            ai["AI Orchestrator<br/>Queue requests, assemble prompts, record statuses"]:::component
            prompts["Prompt Template Registry<br/>Feature-specific prompt versions"]:::component
        end

        subgraph integration["Integration Layer"]
            direction TB
            events["Event Publisher<br/>Document-updated and AI-completed notifications"]:::component
            contracts["Shared Contracts<br/>Zod schemas and DTO validation"]:::component
        end
    end

    web -->|"HTTP requests"| routes
    routes --> auth
    auth --> docs
    auth --> ai
    docs --> policy
    ai --> policy
    docs --> db
    ai --> prompts
    ai --> db
    docs --> events
    ai --> events
    routes --> contracts
    ai --> llm
    events --> hub

    classDef component fill:#EAF4F1,stroke:#2E615D,stroke-width:1.6px,color:#183230;
    classDef external fill:#F9F4EC,stroke:#A88955,stroke-width:1.4px,color:#3B2D0C;
    classDef storage fill:#FFF8F0,stroke:#B78452,stroke-width:1.6px,color:#4B3212;
    style api fill:#FFFFFF,stroke:#2E615D,stroke-width:1.6px,stroke-dasharray: 6 4;
    style edge fill:#FCF8F2,stroke:#8AA6A2,stroke-width:1.2px,stroke-dasharray: 4 3;
    style domain fill:#FCF8F2,stroke:#8AA6A2,stroke-width:1.2px,stroke-dasharray: 4 3;
    style integration fill:#FCF8F2,stroke:#8AA6A2,stroke-width:1.2px,stroke-dasharray: 4 3;
```

A.4 Data Model Diagram

Listing 6: Mermaid source for the data model diagram

```
%%{init: {
  "theme": "base",
  "themeVariables": {
    "fontFamily": "Helvetica, Arial, sans-serif",
    "primaryColor": "#F7F2E8",
    "primaryTextColor": "#183230",
    "primaryBorderColor": "#2E615D",
    "lineColor": "#527A76",
    "secondaryColor": "#EDF4F2",
    "tertiaryColor": "#FFF8F0",
    "background": "#FFFFFF"
  }
}}%%
classDiagram
    direction TB

    class User {
        id: string [PK]
        email: string
        displayName: string
        globalRole: string
        avatarColor: string
    }

    class Session {
        token: string [PK]
        userId: string [FK]
        expiresAt: datetime
    }

    class Document {
        id: string [PK]
        ownerId: string [FK]
        title: string
        content: text
        currentVersion: int
        activeCollaborators: int
        createdAt: datetime
        updatedAt: datetime
    }

    class DocAccess {
        id: string [PK]
        documentId: string [FK]
        principalType: string
        principalId: string
        role: string
        allowAi: bool
        createdAt: datetime
    }

    class DocVersion {
        id: string [PK]
        documentId: string [FK]
        versionNumber: int
        content: text
        createdBy: string [FK]
        reason: string
        createdAt: datetime
    }

    class AiRequest {
        id: string [PK]
        documentId: string [FK]
        initiatedBy: string [FK]
        feature: string
        status: string
        selectionStart: int
        selectionEnd: int
```

```
        sourceText: text
        suggestedText: text
        targetLanguage: string
        requestedVersion: int
        promptTemplateVersion: string
        model: string
        quotaConsumed: int
        createdAt: datetime
        completedAt: datetime
    }

    User "1" --> "0..*" Session : opens
    User "1" --> "0..*" Document : owns
    User "1" --> "0..*" DocVersion : writes
    User "1" --> "0..*" AiRequest : requests
    Document "1" --> "0..*" DocAccess : grants
    Document "1" --> "0..*" DocVersion : records
    Document "1" --> "0..*" AiRequest : tracks
```

A.5 AI Lifecycle Diagram

Listing 7: Mermaid source for the AI lifecycle diagram

```
%%{init: {
  "theme": "base",
  "themeVariables": {
    "fontFamily": "Helvetica, Arial, sans-serif",
    "primaryColor": "#F7F2E8",
    "primaryTextColor": "#183230",
    "primaryBorderColor": "#2E615D",
    "lineColor": "#527A76",
    "secondaryColor": "#EDF4F2",
    "tertiaryColor": "#FFF8F0",
    "background": "#FFFFFF",
    "activationBorderColor": "#2E615D",
    "activationBkgColor": "#EDF4F2",
    "sequenceNumberColor": "#183230",
    "signalColor": "#527A76",
    "signalTextColor": "#183230",
    "labelBoxBkgColor": "#F7F2E8",
    "labelBoxBorderColor": "#2E615D",
    "labelTextColor": "#183230"
  }
}}%%
sequenceDiagram
    autonumber
    actor Writer
    participant Web as Web App
    participant API as Node API
    participant AIS as AI Service
    participant LLM as LLM Provider
    participant RT as Realtime Hub

    Writer->>Web: Select text and request rewrite
    Web->>API: POST /ai-operations<br/>feature + selection + version
    API->>AIS: Validate role, scope context,<br/>create queued interaction
    AIS-->>API: interactionId + queued status
    API-->>Web: 202 Accepted

    AIS->>LLM: Submit scoped prompt<br/>with template version
    LLM-->>AIS: Suggested text
    AIS->>AIS: Persist completed result<br/>and requestedVersion
    AIS->>RT: Emit ai.completed
    RT-->>Web: Push completed interaction
    Web-->>Writer: Show proposal in AI panel

    alt User applies while document version is unchanged
```

```
        Writer->>Web: Apply suggestion
        Web->>API: POST /ai-interactions/:id/apply
        API->>AIS: Verify suggestion is not stale
        AIS->>API: Updated content + new version
        API->>RT: Broadcast document.updated
        RT-->>Web: Push updated document
        Web-->>Writer: Show saved content and history
    else Document changed before apply
        Writer->>Web: Apply stale suggestion
        Web->>API: POST /ai-interactions/:id/apply
        API-->>Web: 409 AI_CONTEXT_STALE
        Web-->>Writer: Prompt reload or fresh AI request
    end
```